Dual-Drone Simulation Framework for Autonomous Coverage and Irrigation in Agricultural Fields

Burra Venkata Chakrapani
B.Tech, Space Science Engineering
Indian Institute of Technology Indore
Email: sse240041005@iiti.ac.in

Bhav Makhija
B.Tech, Space Science Engineering
Indian Institute of Technology Indore
Email: sse240041004@iiti.ac.in

Sana Tejasri Lakshmi
B.Tech, Maths and Computing
Indian Institute of Technology Indore
Email: mc240041033@iiti.ac.in

***Abstract*—This project presents a dual-drone simulation framework for autonomous coverage and irrigation tasks in agricultural fields. Using ROS 2 and PX4 in a Gazebo environment, both drones are designed to collaboratively perform geotagging and irrigation based on randomly assigned severity scores. The drones monitor their own battery and water levels, and dynamically return to the base station for recharging or refilling as needed. A shared coordination allows seamless task handovers and ensures minimal downtime. By implementing intelligent task allocation, role switching, and time-efficiency strategies, the system minimizes total mission duration while maintaining full coverage and effective irrigation. The framework demonstrates how coordinated swarm behavior between just two drones can significantly improve operational efficiency in precision agriculture applications.**

I. Introduction

Agriculture fields often require manual irrigation, which is time-consuming, inefficient, and labor-intensive. Automating this task using drones can improve water efficiency, reduce human effort, and cover large areas faster. In this project, we developed a coordinated dual-drone system that can handle both field coverage and irrigation tasks. Each drone can perform both surveillance and irrigation, and they communicate and collaborate to complete the mission efficiently. The mission planning process is automated using the Lawnmower Generator Node, which takes a KML file containing the field boundary as input and generates a structured JSON file of waypoints following a lawnmower pattern optimized for coverage. The system is fully implemented using ROS 2 (Robot Operating System), PX4 Autopilot, and Gazebo simulation. We created multiple ROS 2 packages to manage drone control, mission logic, recharge/refill handling, geotagging, and waypoint planning. At this stage of the project, both drones have been launched in the Gazebo simulation and are following waypoints using offboard control. The field coverage logic, geotagging, severity scoring, and irrigation triggering based on severity are all implemented. The irrigation drone waits for a duration based on the severity score at each geotag. Battery monitoring and recharge logic have been completed for both drones. Basic swarm coordination and role switching between the drones has been implemented. This report explains the architecture and current progress, along with the next steps planned for completion.

Fig. 1: Irrigation Drone

II. System Architecture

A. Architecture Overview

The dual-drone irrigation system is organized into multiple ROS 2 packages that handle mission logic, drone control, geotagging, swarm coordination, and recharge/refill handling. Each drone operates with its own package (`drone_1_manager` and `drone_2_manager`), which includes nodes for offboard control, geotagging, mission handling, status publishing, drone recharging and water level monitoring. A central ground control station package (`gcs_controller`) supervises both drones, manages waypoint assignment, geotag collection, and role switching logic.The `mission_utils` package includes several utility nodes and datasets essential for mission planning and data management. It contains the **lawnmower_generator_node**, which takes a KML file of the field boundary, generates a lawnmower pattern coverage path, and converts it into a JSON-formatted waypoint file. The **surveillance_monitor_node** monitors the surveillance drone's activity and tracks mission progress. The **geotags_dataset**,**waypoints_dataset** and **geotags_visited_updater_node**are responsible for managing the geotag and waypoint information and updation. Communication between all nodes is standardized using the `drone_interfaces` package, which defines custom ROS 2 messages such as `DroneStatus`, `Waypoint`, `Geotag`. Together, these components form a

modular and scalable architecture that enables coordination between drones for efficient coverage-based irrigation.

B. Architecture Flow and Communication Diagram

The complete architecture integrates ROS 2 nodes, PX4 autopilots, and simulation environments through coordinated information flow.

1) Drone-Level Components: Each drone operates an independent control loop with these core nodes:

- `offboard_controller_node`: Interfaces with PX4 via `/px4_X/fmu/in/trajectory_setpoint` and publishes pose setpoints to `/drone_X/offboard_setpoint_pose`
- `mission_handler_node`: Manages waypoint execution and geotag generation, tracking visited locations
- `geotagging_node`: Generates severity-scored geotags (25m intervals) published to `/drone_X/geotag_generated`
- `status_publisher_node`: Broadcasts drone state via `/drone_X/status`
- `recharge_drone_node`: Monitors `/battery_status` and initiates return procedures
- `water_level_node`:Tracks irrigation water usage, publishes remaining water level to `/drone_X/water_level` and resets the water level upon drone recharge or a role change event.

2) Central Management Nodes:

- `waypoint_manager_node`: Maintains `waypoint_dataset` with visited/unvisited status. Responds to `/waypoint_manager/request`. Distributes waypoint batches via `/drone_X/waypoints`.
- `geotag_manager_node`: Manages `geotag_dataset` with irrigation status. Processes `/geotag_manager/request`. Allocates geotag batches via `/drone_X/geotag_array`.
- `swarm_supervisor_node`: Coordinates role assignment and mission progress. Interfaces with both manager nodes. Implements dynamic role switching logic.
- `dynamic_role_swap_node`: Subscribes to `/drone_X/status`, `/drone_X/water_level`, and `/surveillance_status` and publishes role transitions via `/drone_X/type_change` and `/drone_X/update_status`.

3) Data Flow:

- Mission is planned by `lawnmower_generator_node` to generate waypints JSON file.
- Real-time navigation via PX4 offboard control
- Continuous status monitoring through dedicated topics
- Dynamic task allocation based on: Battery/water levels, Geotag severity scores, Waypoint completion status.
- Persistent state maintenance in JSON datasets

III. Simulation Setup and Tools Used

The dual-drone irrigation framework has been developed entirely in a simulated environment using Gazebo, PX4 Autopilot, and ROS 2. The simulation is based on the iris model, which provides a realistic drone configuration with support for offboard control via MAVLink communication. Each drone operates in its own namespace, with PX4 running in Software-In-The-Loop (SITL) mode to simulate flight behavior. ROS 2 Humble is used as the communication backbone between drone control nodes, ground station coordination nodes, and supporting utilities. MAVLink protocol is used to interface with the PX4 autopilot for sending trajectory setpoints and controlling drone behavior in offboard mode. QGroundControl (QGC) is used alongside the system to monitor flight status and manage the PX4 parameters when needed. Waypoints for the mission are first designed using `.kml` files, which are then converted to JSON format using `converter.py` script. The JSON files contain geodetic waypoints that are converted to local NED coordinates dynamically within the nodes, based on the home position received from PX4 during simulation. This flexible conversion approach ensures that the drones can operate relative to their actual spawn location in the Gazebo world. The entire system is tested on Ubuntu with ROS 2 Humble and PX4 running in SITL mode.

IV. Workflow

The workflow of the dual-drone coordinated system is organized into six clear phases, each representing a functional segment of the mission. The system ensures full autonomy in coverage and irrigation while incorporating role switching, resource monitoring, and task coordination.

A. Phase 1: Mission Initialization and Preprocessing

- A `.kml` file representing the field layout is provided and then loaded into `QGroundControl` (QGC), where a lawnmower-style coverage path is manually designed for optimal field traversal.
- The `.kml` file is exported from QGC and processed using the `converted.py` script located in the `scripts/` directory, which converts it into a JSON file containing structured waypoint data, including latitude, longitude, altitude, and visited flags.
- The generated JSON file is loaded by both the `waypoint_manager_node` (for surveillance tasks) and the `geotag_manager_node` (for irrigation tasks). These nodes prepare and manage the dispatching of task batches to the drones.
- The `swarm_supervisor_node` assigns roles (surveillance or irrigation) and directions (top or bottom) to the drones, and sends requests to waypoint manager node to send waypoints batches.
- Once batches are assigned, each drone's `mission_handler_node` begins the mission by commanding takeoff through its `offboard_controller_node`. After reaching takeoff altitude, the drone starts navigating

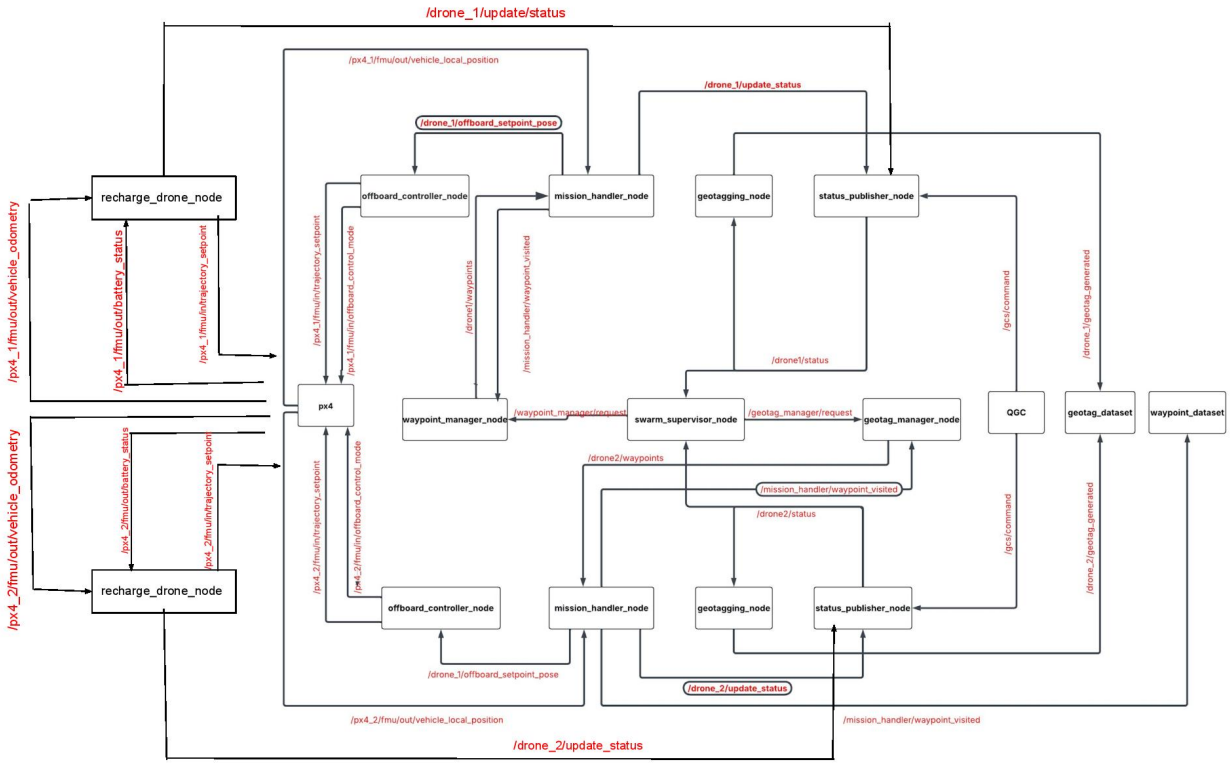


Fig. 2: System architecture diagram showing ROS 2 nodes and communication flow between components.

to the first assigned waypoint or geotag setpoint, initiating the coverage or irrigation phase based on its role.

B. Phase 2: Coverage and Geotagging

- Surveillance-type drones receive unvisited waypoint batches from the `waypoint_manager_node`, in response to `WaypointRequest` messages sent by the `swarm_supervisor_node`.
- These batches are direction-specific (top/bottom) and published on the topic `/drone_x/waypoints` using `WaypointArray` messages.
- The `mission_handler_node` navigates through the assigned waypoints using offboard setpoints, while simultaneously checking for completion of each waypoint via Euclidean distance.
- During navigation, the `geotagging_node` generates geotags every 25 meters, simulating GPS-tagged crop points with randomized severity scores.
- Each geotag (with severity score) is published to `/drone_x/geotag_generated` and collected by the `geotag_dataset` node.
- These geotags are marked as `irrigated: false` and saved to a central JSON file for irrigation assignment.

C. Phase 3: Irrigation Task Assignment

- The `swarm_supervisor_node` sends `GeotagRequest` messages to the `geotag_manager_node` to obtain batches of unirrigated geotags.
- These batches are filtered by direction and severity status and sent to irrigation drones via the `/drone_x/geotag_array` topic as `GeotagArray` messages.
- The irrigation drone's `mission_handler_node` loads these geotags, converts them to local NED coordinates, and prepares for sequential irrigation.

D. Phase 4: Irrigation Execution (Severity-Aware)

- The drone navigates to each geotag using dynamic offboard setpoints published to `/drone_x/offboard_setpoint_pose`.
- Upon reaching the target geotag, the drone waits for `severity_score` $\times$ `10` seconds to simulate irrigation time — higher severity results in longer irrigation.
- After the wait, it publishes a `WaypointVisited` message and the `geotag_dataset` is updated to mark the point as irrigated.
- The mission handler then proceeds to the next unirrigated geotag in the list.

E. Phase 5: Battery and Water Monitoring with Recharge Handling

- Each drone runs a `recharge_drone_node` that monitors battery drain rate and water usage in real time via `/battery_status` and `/vehicle_odometry`.

- When a threshold is reached (based on return feasibility), the drone is marked as returning using a `DroneStatusUpdate` message.
- The `status_publisher_node` disables waypoint/geotag tracking and redirects the drone to the recharge/refill station using offboard setpoints.
- Once charging/refilling is complete (simulated via Gazebo battery overrides), the drone is reactivated and resumes its task from the last geotag or waypoint.

F. Phase 6: Dynamic Role Switching and Mission Completion

- If one drone is unavailable due to recharge/refill, the other drone dynamically takes over its role (surveillance or irrigation) to minimize idle time.
- The `swarm_supervisor_node` monitors drone status from `/drone_1/status` and `/drone_2/status` and dynamically reassigns roles and batches as needed.
- The mission continues until:
  - All waypoints are marked visited in the waypoint JSON.
  - All geotags are marked irrigated in the geotag JSON.
- Once the mission is complete, the GCS terminates the operation and both drones return to base.

V. Node-Level Explanation

A. Drone_1_Manager and Drone_2_Manager Nodes

1) Offboard Controller Node: The `offboard_controller_node.py` node is responsible for managing the offboard flight control pipeline for each drone using the PX4 flight stack via ROS 2. It interfaces with the PX4 Autopilot by publishing to and subscribing from MAVLink-bridged PX4 topics under the `/px4_1/...` (Drone 1) and `/px4_2/...` (Drone 2) namespaces, respectively. This node initializes and continuously sends `OffboardControlMode` messages at 10 Hz to maintain offboard control heartbeat. It also publishes position-based `TrajectorySetpoint` messages that guide the drone's movement in the local NED (North-East-Down) frame. Vehicle arming, mode switching to `OFFBOARD`, and landing commands are issued through the `VehicleCommand` topic using MAVLink command IDs (`VEHICLE_CMD_COMPONENT_ARM_DISARM`, `VEHICLE_CMD_DO_SET_MODE`, and `VEHICLE_CMD_NAV_LAND`). The node subscribes to the following PX4 telemetry topics to track the drone's state:

- `/fmu/out/vehicle_local_position` for the local frame position
- `/fmu/out/vehicle_odometry` for pose and velocity
- `/fmu/out/vehicle_status` for navigation mode detection
- `/fmu/out/vehicle_command_ack` for PX4 command acknowledgements
- `/px4_1/fmu/out/vehicle_land_detected` for landing detection

After initialization, the node performs a staged takeoff by publishing an initial position setpoint with a fixed Z-axis offset (`takeoff_height`). Once the takeoff height is reached and the PX4 navigation state confirms `OFFBOARD` mode, a flag (`self.taken_off`) is set to enable real-time waypoint tracking. The node then begins tracking incoming dynamic setpoints of type `PoseStamped` from the topic `/drone_1/offboard_setpoint_pose` or `/drone_2/offboard_setpoint_pose` (depending on the drone). Upon receiving a dynamic waypoint, it calculates a heading (yaw) toward the target using the current local position and publishes the corresponding setpoint. Yaw computation is performed using `arctan2` over the position delta (`dx`, `dy`), ensuring the drone aligns its heading with its motion vector during waypoint transition.

he offboard control node is responsible for managing the drone's flight based on its current status. When the drone's status changes to "charging," the node immediately initiates a landing sequence to safely bring it down at the base station and disarm it.

Once the drone is fully charged, its status updates to "charged." The offboard node then triggers a takeoff sequence to get the drone back into the air. After a successful takeoff, the drone's status is set to "idle," signaling that it is ready to receive new commands and continue its mission.

This node serves as the final actuator interface that converts high-level navigation decisions into PX4-consumable trajectory and command messages.

2) Mission Handler Node: The `MissionHandlerNode` is a ROS 2 node responsible for managing waypoint navigation and mission execution for both `drone_1` and `drone_2`, adapting its behavior based on the drone's role as either surveillance or irrigation. Both nodes follow the same internal logic and structure with only two key differences: the drone ID (`drone_1` vs `drone_2`) and the PX4 local position topic (`/px4_1/fmu/out/vehicle_local_position` vs `/px4_2/fmu/out/vehicle_local_position`). The node subscribes to the topic `/drone_x/status` (`DroneStatus` message) to detect the drone's type. Upon receiving this information, it conditionally subscribes to either `/drone/drone_x/waypoints` (`WaypointArray`) if the type is surveillance, or `/drone_x/geotag_array` (`GeotagArray`) if the type is irrigation. In both cases, the received waypoints or geotags are stored in an internal list, and severity scores are stored in a dictionary when the drone is in irrigation mode. The geodetic coordinates of waypoints are transformed to local NED (North-East-Down) coordinates using the `pyproj` library. The node waits for PX4 to publish valid GPS origin values on the topic `/px4_x/fmu/out/vehicle_local_position`, from which the reference latitude, longitude, and altitude are extracted and used to define the NED projection frame. Once the origin is set, the node begins publishing `PoseStamped` messages to `/drone_x/offboard_setpoint_pose`, which act as offboard control setpoints that PX4 uses to move the drone toward the target waypoint. Waypoint progression is handled using the `_publish_-`

next_waypoint() function. The node calculates the Euclidean distance between the current drone position (from PX4) and the target waypoint. If the distance is less than a 1-meter threshold, it considers the waypoint reached. In surveillance mode, the drone proceeds to the next point immediately. In irrigation mode, the node waits for a duration proportional to the severity score (severity $\times$ 10 seconds) before proceeding to the next waypoint representing sprinkling the water. After this wait, a WaypointVisited message is published on /mission_handler/waypoint_visited to inform other systems that the current point has been covered. Additionally, the node continuously updates the mission status by publishing DroneStatusUpdate messages on /drone_x/update_status to reflect whether the drone is idle, executing, or has completed its mission. This node plays a central role in connecting high-level mission logic (waypoint and severity handling) with low-level PX4 control through the offboard interface, enabling precise, role-dependent autonomous behavior.

3) Geotagging Node: The GeotaggingNode is responsible for generating geotags with GPS coordinates during surveillance missions based on the drone's movement. It runs separately on both drones (drone_1 and drone_2) with respective changes in topic names. This node subscribes to two key topics: /drone_X/status (of type DroneStatus) to detect if the drone is actively performing a surveillance mission, and PX4's /px4_X/fmu/out/vehicle_local_position (of type VehicleLocalPosition) to receive continuous local position data. The geotagging operation is enabled only when the drone status is 'executing' and type is 'surveillance'. Once activated, the node checks for valid GPS reference data (ref_lat, ref_lon, ref_alt) to define the local NED (North-East-Down) projection system using the pyproj library. A geotag is generated in a way that the distance between them is 25 m to ensure that a large number of geotags are not generated in very short distance and represent real world scenario. A random severity_score between 1 and 5 is assigned to simulate crop stress levels. This geotag is then published on the /drone_X/geotag_generated topic. The function save_geotag() handles the conversion and message population, while a helper euclidean() function calculates the distance moved. Both versions (Drone 1 and Drone 2) share the same core logic, differing only in the topic prefixes (e.g., /drone_1 vs /drone_2, and /px4_1 vs /px4_2).

The initial logic for geotagging had a key flaw: the geotagging node was active as soon as the drone took off. This resulted in the unintentional generation of geotags near the base station, as the drone traveled from its origin to the agricultural field. These "unwanted geotags" cluttered the mission data and introduced inaccuracies, as they didn't correspond to actual locations within the coverage area.

To solve this, we implemented a flag-based logic to precisely control when geotagging begins. This flag is initially set to false. The drone proceeds with its mission, but the geotagging node remains inactive. The flag only becomes true when the drone successfully reaches the first designated waypoint of the lawnmower pattern inside the field. From this point on, geotag generation is enabled. This ensures that all collected geotags are relevant to the agricultural field and eliminates the creation of inaccurate data near the origin. This simple yet effective solution makes the system more robust and ensures the integrity of the data collected during the mission.

Overall, this node plays a key role in dynamically marking geospatial crop data during surveillance, enabling targeted irrigation in later stages based on severity scores.

4) Recharge Drone Node: The Recharge_drone_node continuously monitors multiple real-time parameters including battery status from the /battery_status topic, current water level, and the drone's relative position from the recharge station via /Vehicle_odometry. These inputs feed into a logic function that calculates both the required battery percentage needed to safely return to the recharge station (using drain rate calculations from battery telemetry) and the precise threshold level at which to initiate the return sequence. The node then publishes status updates through the /update_status topic to the Status_publisher_node, which acts as a system-wide state switch capable of overriding other nodes' operations. When the returning_to_recharge status becomes active, this override mechanism interrupts the Mission_handler_node, immediately halting all waypoint reception and commanding the drone to navigate back to the recharge station via shortest-path trajectory updates published to /drone_x/offboard_setpoint_pose. During the recharge phase, the node directly manipulates Gazebo's battery_status_simulation parameters to control charging rates and accurately simulate recharge durations, while simultaneously preserving the last visited waypoint in temporary memory. Upon completing the recharge cycle and receiving the resuming_work status confirmation from the Status_publisher_node, the system seamlessly transitions control back to the Mission_handler_node, allowing the drone to resume operations precisely at its previously interrupted waypoint. This intelligent discharge-recharge cycle continues iteratively until all assigned geotagging and irrigation tasks are fully completed, ensuring uninterrupted mission execution while maintaining optimal battery management.To avoid conflicts with the dynamic role swap, we created a surveillance monitoring node which publishes if surveillance and irrigation is completed or not. This is done to avoid returning for recharge in case water_level has reached zero but some amount of battery is still there, in this case the drone is commanded to change its role to surveillance and continue surveillance from the opposite direction.

5) Water Level Node: The Water Level Node is responsible for publishing the /water_level topic that represents the percentage availability of water in the tank (reducing from 100 to 0). The node subscribes to geotag_visited messages to know which geotag is being irrigated in real time.

Along with this, it also receives the severity score of each geotag in a batchwise order from the previously published `geotag_array` messages. Based on the severity score of each geotag, a certain percentage of water (in proportion to severity) is released at that geotag, which can be monitored using the `/water_level` topic.To make it more efficient for a dual-drone system, the node also communicates with the drone's status and type (via `DroneStatus` and `DroneType` messages) to identify which drone needs to be refilled and at what time recharge should be triggered, while ensuring the surveillance drone continues unaffected. This also enables the node to handle refills in case the role swap logic is activated.

6) Status Publisher Node: The `StatusPublisher` node is a core component responsible for broadcasting each drone's current mission state (`idle`, `executing`, etc.) and role (`surveillance` or `irrigation`) at regular intervals. This node operates independently in both `drone_1` and `drone_2` with topic-level customization based on the drone ID. It publishes `DroneStatus` messages on the `/drone_X/status` topic (where X is 1 or 2) and provides a centralized source of truth for the drone's status that can be used by mission handlers, geotagging, GCS, and monitoring systems. The node subscribes to `/qgc/command`, which carries `DroneTypeChange` messages from the Ground Control Station (QGC). These messages are used to dynamically update the drone's type (e.g., switching from surveillance to irrigation). Additionally, it listens to `/drone_X/update_status`, which receives `DroneStatusUpdate` messages from the mission handler node and the recharge drone node whenever the drone's task phase changes (like from `idle` to `executing`). Internally, it maintains two variables: `drone_status` and `drone_type`, which are updated based on incoming messages and are published every 2 seconds using a timer. The main callback `qgc_command_callback()` updates the drone type, and `mission_status_callback()` adjusts the current mission status. The `publish_drone_status()` function then wraps these values into a `DroneStatus` message and publishes them periodically. While both drone versions of the node share the same logic, `drone_1` starts with `surveillance` as the default type, and `drone_2` starts with `irrigation`. This node plays a vital role in synchronizing drone state across all components of the system and ensuring smooth transitions between mission phases.

B. Drone Status Definitions

To enable effective coordination, state monitoring, and decision-making in the multi-drone system, each drone operates under a well-defined status framework. The status reflects the current phase of the drone's operation within the irrigation mission and is dynamically updated throughout the mission lifecycle. The defined statuses are:

- **Idle:** The drone is in a standby state and is ready to receive a new batch of waypoints. It has completed any previous mission or has just been initialized.
- **Executing:** The drone is actively flying and following a batch of assigned waypoints for irrigation or surveillance tasks. It remains in this state until the batch is completed or an interrupt condition (e.g., low battery or water) occurs.
- **RTL (Return-To-Launch):** The drone transitions to this state when it needs to return to the origin point, either for battery recharging or water refilling. The return is initiated based on threshold conditions to ensure safe and efficient recovery.
- **Charging:** Upon reaching the origin or charging station, the drone begins its charging process. During this phase, the drone is grounded and not available for mission tasks.
- **Charged:** The drone has completed its charging cycle and is ready for takeoff. It remains in this state until a new waypoint batch is assigned and acknowledged.

C. GCS Controller

1) Swarm Supervisor Node: The `SwarmSupervisorNode` plays a central role in managing and coordinating the swarm behavior of the two drones. It listens to status updates from both drones via the `/drone_1/status` and `/drone_2/status` topics, which publish `DroneStatus` messages. Based on these updates, the node decides when and how to dispatch drones for their respective missions — either coverage (surveillance) or irrigation — while ensuring they work in coordination. Each drone is tracked using internal dictionaries `drone_status` and `last_seen`, where their current mode, status, and last active timestamp are stored. If a drone becomes idle and is not yet actively assigned to a task, the node attempts to pair it with the other idle drone (if available) to enable simultaneous and non-overlapping execution of tasks. If both drones are of the same type, they are assigned opposite directions (top and bottom) for efficient field division. If they are of different types, each is dispatched based on its current assigned role and direction. Dispatching requests is done using two ROS 2 publishers: `/waypoint_manager/request` for surveillance missions using `WaypointRequest` messages, and `/geotag_manager/request` for irrigation missions using `GeotagRequest` messages. These messages contain the `drone_id` and the direction (top/bottom), guiding the next step in mission assignment. To prevent inactive drones from blocking operations, the node uses a periodic timer to check for unpaired drones. If one drone remains idle and the other drone is either offline (not updated for more than 10 seconds) or currently active, the idle drone is dispatched solo. This prevents delays in coverage or irrigation due to temporary loss or unavailability of the partner drone. The function `status_callback` is triggered whenever a status message is received. It logs the status and type, updates tracking variables, and evaluates if the drone should be dispatched. The function `check_for_unpaired_drones` runs every 2 seconds to evaluate drones that might need to be dispatched alone. The `send_request` function handles message publication to the correct topic based on drone

type. This node acts as the swarm-level mission coordinator ensuring fault-tolerant, synchronized behavior across both drones. It supports dynamic role switching, timeout-based fallback logic, and ensures optimal use of available drone resources.

2) Waypoint Manager Node: The `WaypointPublisher` node handles the distribution of unvisited waypoints to surveillance drones in batch form, based on direction and request. It begins by loading a JSON file specified via the `json_file_path` parameter. Each waypoint entry in the file must include `latitude`, `longitude`, `altitude`, `ID`, and a boolean `visited` status. Once loaded, the node creates two types of waypoint batches: `ascending` batches (for the `top` direction) and `descending` batches (for the `bottom` direction), each of fixed `batch_size` configured via parameters. The node subscribes to two topics: `/waypoint_manager/request` for `WaypointRequest` messages, and `/mission_handler/waypoint_visited` for `WaypointVisited` messages. Upon receiving a request, the node filters and selects a batch of unvisited waypoints for the given direction. It ensures that the same batch is not assigned to multiple drones by tracking assignments through an internal dictionary. Waypoints that are already visited are skipped, and each eligible batch is published to a dynamically generated topic in the format `/drone/{drone_id}/waypoints` using the `WaypointArray` message type.

3) Geotag Manager Node: The `GeotagNode` manages the distribution of geotag batches to irrigation drones upon request. It processes data stored in a JSON file containing geotag entries, each with GPS coordinates, altitude, severity score, and irrigation status markers. The node actively monitors the `/geotag_manager/request` topic for incoming `GeotagRequest` messages from the Swarm Supervisor Node, which trigger irrigation missions. These request messages include both a `drone_id` identifier and a `direction` field (either `top` or `bottom`), used for systematic geotag allocation and field division. Upon receiving a request, the node loads all unirrigated geotags from the specified JSON file by filtering for entries marked with `"irrigated": false`. The system then sorts these geotags - in ascending order of ID for `top` direction requests and descending order for `bottom` direction - creating an efficient spatial work distribution pattern across the agricultural field. A configurable `batch_size` parameter determines how many geotags are included in each response, allowing flexible mission segmentation. The node maintains precise batch tracking through a `sent_indices` dictionary that records progression for each (`drone_id`, `direction`) combination. This persistent tracking ensures subsequent requests continue from the correct position in the geotag sequence. When preparing a batch, the node packages the selected geotags into a `GeotagArray` message and publishes it to a dynamically generated topic following the pattern `/{drone_id}/geotag_array`, creating dedicated communication channels for each drone. This design separates mission generation from real-time decision making while leveraging JSON for persistent state management. The architecture provides reliable, scalable irrigation task distribution in multi-drone operations, with deterministic geotag allocation ensuring complete field coverage without duplication of effort between cooperating drones.

4) Dynamic Role Swap Node: The `DynamicRoleSwapNode` changes drone roles between surveillance and irrigation based on real-time status updates and mission progress. It manages both drones, subscribes to each of their status updates on the topic `/<drone_id>/status` using the custom message type `DroneStatus`, water levels on `/<drone_id>/water_level` with standard `Float32` messages, and overall surveillance mission status from the topic `/surveillance_status` using the custom `SurveillanceStatus` message. It publishes type-change commands using the topic `/<drone_id>/type_change` with `DroneTypeChange` messages, and updates drone status via `/<drone_id>/update_status` using `DroneStatusUpdate` messages.

Two key conditions trigger dynamic role changes:

1) Low water level during irrigation (below a threshold) and surveillance not yet complete, leading to a swap to surveillance.
2) Surveillance completion, prompting all surveillance drones to become irrigation drones.

This ensures that drone resources are efficiently reused without manual intervention, based on status-aware logic and synchronized topic-based communication. This swap is carried out by the `swap_drone_role()` method, which internally calls `publish_type_change()` to change the drone's role and `publish_status_update()` to update status.

D. Mission Utils

1) Lawnmower Generator Node: The `Lawnmower Generator Node` helps in generating efficient lawnmower coverage paths over a polygonal area defined through a KML file. It begins by subscribing to the PX4 vehicle local position topic to receive the drone's home GPS coordinates. The node parses the boundary from the KML file and converts the GPS coordinates into projected XY coordinates. It then rotates the area such that the passes align parallel to the longest side of the polygon, minimizing the number of turns and improving time efficiency. Based on the specified spacing and turnaround parameters, it generates lawnmower lines using `shapely` geometries, and selects the farthest endpoint from the home location to determine the optimal start direction. The XY waypoints are converted back to GPS and saved in a structured JSON file. Additionally, the node automatically deletes any old mission JSON files to ensure clean generation. This node automates the creation of mission waypoints, eliminating the need for manual planning in `QGroundControl` and saving significant setup time.

2) Geotag Dataset: The `geotag_dataset` node collects and stores all the geotags generated by Drone 1 during its surveillance mission. This node subscribes to the `/drone_1/geotag_generated` topic, which sends geotag messages containing the GPS coordinates (`latitude`, `longitude`, `altitude`), `severity score`, and a unique `ID` for each detected point in the field. Every time a new geotag is received, the node adds it to a list and writes the updated list into a JSON file. The path to this file can be set using a launch parameter called `geotag_file_path`, and by default, it saves to `geotags.json`. An `irrigated` field is added to each geotag and initialized to `False`, so we can later update this status after irrigation is done. This collected dataset helps the rest of the system, especially during the irrigation phase, to decide which locations need to be serviced and which ones are already done.

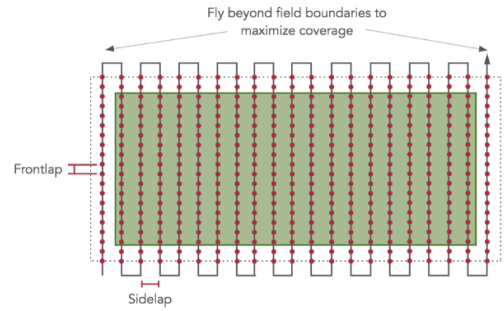
3) Geotags Visited Updater Node: The `Geotag Updater Node` is responsible for marking geotag locations as irrigated based on feedback received from drones. It subscribes to the `/mission_handler/geotag_visited` topic, which carries custom `GeotagVisited` messages containing the geotag ID and drone ID. Upon receiving a message, the node safely accesses the geotag JSON file, updates the corresponding geotag's `irrigated` field to `true`, and saves the changes. It ensures file consistency by writing to a temporary file and replacing the original only after a successful update.

4) Waypoint Visited Updater Node: The `WaypointVisitedUpdater` node is a crucial part of the mission management system. Its primary function is to track the progress of the autonomous drone mission by monitoring which waypoints have been visited. It subscribes to the /mission_handler/waypoint_visited topic, listening for messages from the drone's mission controller. When a WaypointVisited message is received, the node extracts the waypoint's ID and then updates a shared JSON file, which serves as the mission's central data store. This update marks the corresponding waypoint as "visited," ensuring that the system has an up-to-date and persistent record of mission progress. This mechanism is essential for a robust and fault-tolerant system, as it allows for seamless task handovers and prevents drones from revisiting locations that have already been covered.

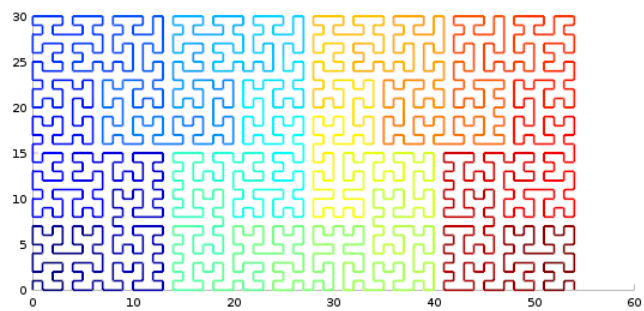
5) Surveillance Monitor Node: The **Surveillance Monitor Node** tracks the progress of the drone's surveillance mission by reading a JSON file containing waypoints. It checks how many waypoints remain unvisited and publishes this information using a custom `SurveillanceStatus` message, which includes a completion flag and the remaining waypoint count.

VI. Key Innovations in Dual Drone Framework

A. Why the Simple Lawnmower Path Is the Optimal Choice for Our Dual-Drone Framework



(a) Lawnmower coverage pattern



(b) Hilbert curve pattern for large swarm

Fig. 3: Comparison of coverage strategies used by irrigation and surveillance drones

1) Optimized for Small Drone Teams, Not Massive Swarms: Advanced multi-agent path planning algorithms such as Hilbert curves, Rapidly-Exploring Random Trees (RRT), and multi-agent A* are primarily designed for complex, dynamic, or cluttered environments involving large fleets of lightweight swarm drones. While powerful, these algorithms introduce unnecessary computational overhead and coordination complexity for small-scale systems.

In our framework, which utilizes only two high-capacity drones—one dedicated to irrigation and the other to surveillance—such algorithms are overkill. The operational environment consists of structured, open agricultural fields where deterministic and repeatable coverage patterns are sufficient.

As illustrated in **Figure 3a** and **Figure 3b**, we rely on simpler yet effective coverage strategies. The lawnmower pattern (Figure 3a) provides linear, full-field coverage with minimal turns, making it ideal for efficient irrigation. In contrast, the Hilbert curve (Figure 3b) is used when the field is split into multiple zones and requires localized, recursive coverage—especially useful for detailed surveillance tasks. These patterns ensure high efficiency, reduced inter-drone conflict, and minimal coordination overhead while fully leveraging the capabilities of each drone.

2) Optimized for High-Payload Drones with Minimal Turns: Our irrigation drones carry significant payloads (water tanks),

E. Drone Interfaces

TABLE I: ROS2 Messages

Message/Service	Fields	Topic	Publisher Node	Subscriber Node
`DroneStatus.msg`	drone_id, type, status, direction	`/drone_1/status`, `/drone_2/status`	StatusPublisher	SwarmSupervisor
`DroneStatusUpdate.msg`	drone_id, status	`/drone_1/update_status`, `/drone_2/update_status`	MissionHandler	StatusPublisher
`DroneTypeChange.msg`	new_drone_type	`/qgc/command`	QGC (external system)	StatusPublisher
`Geotag.msg`	id, lat, lon, alt, severity_score	`/drone_1/geotag_generated`, `/drone_2/geotag_generated`	GeotaggingNode	Geotag_dataset
`GeotagArray.msg`	geotags (list of Geotag)	`/drone_1/geotag_array`, `/drone_2/geotag_array`	GeotaggingNode	MissionHandler
`GeotagRequest.msg`	drone_id, direction	`/geotag_manager/request`	SwarmSupervisor	GeotaggingNode
`GeotagVisited.msg`	drone_id, geotag_id	`/mission_handler/geotag_visited'`	mission_handler_node	geotags_visited_updater node
`SurveillanceStatus.msg`	surveillance_completed, waypoints_remaining	`surveillance_status`	surveillance_monitor_node	dynamic_role_swap_node
`Waypoint.msg`	id, lat, lon, alt, visited	Embedded in WaypointArray	WaypointPublisher	MissionHandler
`WaypointArray.msg`	waypoints (list of Waypoint)	`/drone/{id}/waypoints`	WaypointPublisher	MissionHandler
`WaypointRequest.msg`	drone_id, direction	`/waypoint_manager/request`	SwarmSupervisor	WaypointPublisher
`WaypointVisited.msg`	waypoint_id, drone_id	`/mission_handler/waypoint_visited`	MissionHandler	WaypointPublisher

which make sharp and frequent turns inefficient and battery-intensive. Complex patterns usually involve quick direction changes, beneficial for lightweight UAVs, but counterproductive for our scenario. The lawnmower path minimizes directional changes, allowing our heavy-duty drones to operate in long, straight, fuel-efficient sweeps with better water discharge stability and less pressure on drone components over time.

3) Turns = Battery Drain + Time Loss: Every turn a drone makes isn't just a change in direction; it demands extra thrust, time, and energy—especially for loaded irrigation drones. These maneuvers significantly drain the battery, introduce precision delays, and disrupt mission continuity. Lawnmower patterns reduce this overhead drastically by following linear paths with consistent spacing, enabling smoother motor performance and faster mission execution.

In mission-critical tasks like irrigation, where both battery conservation and time efficiency are paramount, this straightforward pattern proves to be not just simple, but **superior**.

B. Dynamic Mass Variation

To realistically simulate payload-dependent drone dynamics, we have implemented a dynamic mass variation mechanism driven primarily by the drone's onboard water level. Since the water tank constitutes a significant portion of the drone's payload, fluctuations in water volume during irrigation missions have a direct and substantial impact on the overall weight of the drone.

To capture this behavior accurately in simulation, we integrated a custom Gazebo plugin that listens to the `/water_level` topic, which is continuously updated by our water monitoring subsystem. This plugin interprets the real-time water level data and adjusts the drone's mass accordingly within the physics engine.

As a result, the plugin modifies the drone's physical characteristics such as inertia, required thrust for hovering, and translational acceleration in response to the dynamic changes in weight. This ensures that drones carrying a full water tank behave differently from those with an empty one, enabling a more precise and realistic simulation of flight dynamics under varying payload conditions. This is especially critical for irrigation tasks where water depletion over time drastically influences flight performance and control behavior.

VII. Time Efficiency and Coordination Strategy

Several strategies are implemented in the dual-drone system to ensure efficient use of time and resources during the mission:

- **Optimized Path Planning:** Instead of relying on a simple pre-planned mission, we developed a custom Lawnmower Generator Node. This node takes a KML file of the field boundary and intelligently creates a "lawnmower pattern" of waypoints. By generating these waypoints from the farthest to the nearest points in the field, we ensure a systematic and efficient coverage path. This approach minimizes redundant travel and turns, significantly improving mission speed and conserving battery life.
- **Dual-Start Strategy:** Both surveillance and irrigation drones start from the farthest point from the take-off location, with the surveillance drone ending its path near the base. This allows the converted irrigation drone to begin irrigation close to the station, minimizing travel time for refilling and saving significant energy.
- **Intelligent Return Logic:** An adaptive and context-aware return mechanism is implemented to optimize drone efficiency during irrigation missions. The system continuously monitors both battery level and onboard water level in real time. When the battery level drops to a threshold where only a safe return to the home/origin position is feasible, the drone initiates an immediate return, avoiding longer or energy-intensive paths to prevent mission failure. However, if the water level is depleted but sufficient battery remains, the drone does not return

immediately; instead, it intelligently switches its role to perform surveillance operations over the field. This ensures that no time or battery is wasted during transit without active irrigation, thereby improving mission effectiveness by utilizing remaining energy for productive tasks.

- **Role Flexibility:** Both drones can perform coverage and irrigation tasks. If one goes for recharge or refill, the other takes over its role, minimizing idle time.
- **Severity-Based Prioritization:** Geotags with higher severity scores are prioritized for irrigation, and delay logic is applied based on the score to avoid servicing low-priority points early.
- **Redundancy Avoidance:** Each geotag includes an `irrigated` flag stored in a JSON file. This prevents drones from revisiting already irrigated points, saving mission time.
- **Real-Time Status Updates:** Drones publish their battery, water, and position status continuously. This allows dynamic reassignment of tasks in case one drone becomes unavailable.
- **Parallel Processing:** Geotags are generated in real-time while drones follow the coverage path, eliminating the need for a separate geotagging phase.
- **Efficient Waypoint Tracking:** The `WaypointVisitedUpdater` marks visited waypoints during runtime, ensuring that drones don't revisit completed points unnecessarily.
- **Coordinated Recharging:** Implemented recharge queuing to avoid simultaneous discharge and time-efficient recharging
- **Intelligent Energy Profiling:**The system models a dynamically adaptive battery discharge rate for the irrigation drone, coupled with the real-time water level in its tank. As irrigation reduces the payload weight, the drone consumes less energy per unit distance enabling extended flight durations and smarter energy budgeting. This physics informed adjustment not only maximizes operational endurance but also enhances decision-making for refilling and return-to-base triggers, showcasing a deep integration of physical dynamics into system intelligence.

Key Technical Innovations and Special Features

- **Variable Mass and Speed Modeling**
  Our framework accounts for changing drone mass during the mission—especially as water is consumed for irrigation. This adds realism to the simulation, affecting both drone dynamics and energy usage. Drone speed is also adjustable based on current payload, mission phase, and severity-based irrigation requirements, making the simulation closer to real-world performance.
- **Dynamic Battery Discharge Simulation**
  We implemented a dynamic battery discharge mechanism that varies energy consumption based on drone activity. For example, flying while carrying a full water tank or irrigating at multiple geotags consumes more battery than idle hovering or surveillance. This enhances mission planning and resource-aware role switching.
- **Partial Integration Due to Time Constraints**
  While the battery discharge model was initiated and partially integrated with different drone states and resource levels, full implementation remains incomplete. There are known issues with syncing discharge rates dynamically across all conditions and states, especially under simultaneous irrigation and movement. Due to time constraints and prioritization of core functionalities, this component remains under development and offers a clear direction for future enhancement.
- **Realistic Energy-Resource Tradeoffs**
  By introducing elements like variable mass, role-specific power consumption, and partial battery logic, our project stands apart from standard simulations. It lays the groundwork for advanced mission scheduling based on real-time drone limitations, encouraging future extensions involving power-aware path planning and docking logic.

VIII. Conclusion

This project successfully establishes a robust and scalable dual-drone simulation framework for autonomous coverage and irrigation in agricultural fields. By leveraging the modularity of ROS 2, combined with the powerful PX4 autopilot and Gazebo simulation, we have created a system that demonstrates significant improvements in operational efficiency over traditional, manual methods.

The core of our framework lies in its intelligent coordination and highly modularized architecture. We've implemented a centralized ground control station that facilitates dynamic task allocation, seamless role switching between drones, and efficient mission handovers, all of which are critical for minimizing downtime and maximizing coverage. The system's ability to autonomously manage battery levels and water supplies ensures that the drones can return to a base station for recharging or refilling as needed, making the entire operation self-sufficient.

Furthermore, the integration of a Lawnmower Generator Node and a custom geotagging system provides a structured, time-efficient approach to field coverage and targeted irrigation. The system's intelligence is showcased in its ability to assign severity scores and trigger irrigation based on specific needs, ensuring water is used efficiently and precisely.

Ultimately, this dual-drone framework is more than just a simulation; it's a proof of concept for the future of precision agriculture, designed with real-world applications in mind. The system's core design principles—autonomy, dynamic task allocation, and resource management—address key challenges faced in large-scale agricultural operations. By demonstrating the effectiveness of a coordinated two-drone swarm in a simulated environment, we confirm its potential to be a robust, time-efficient, and highly scalable solution for enhancing productivity and resource management in real-world agricultural settings.

IX. Future Scope

The current simulation framework lays the foundation for an intelligent and coordinated dual-drone system for coverage-based irrigation. However, there's scope to extend this project into real-world applications and cutting-edge research directions. we are very excited to deploy this framework on real drones equipped with GPS, water sprayers, and onboard sensors. Testing in actual agricultural environments will help validate system performance under real-world conditions like uneven terrain, wind disturbances, and fluctuating GPS accuracy. Currently the framework is designed for two drones but can be scaled to a full swarm of autonomous drones. Advanced coordination strategies like decentralized decision-making, role switching, and dynamic task reallocation can enable efficient coverage of large areas with minimal overlap, even in the event of individual drone failures. Integrating AI/ML models can significantly boost system intelligence. Drones can detect crop types, assess plant health, and prioritize irrigation based on real-time camera input using deep learning models like YOLO or segmentation networks. This turns the drones into smart, adaptive agents capable of precision agriculture. To better prepare for real-world deployment, the simulation can be enhanced with factors like soil moisture dynamics, elevation maps, weather effects, and crop-specific water requirements. This allows for more robust testing and training of drone behavior under realistic farm conditions.

References

The code and documentation are available at GitHub Repository:
https://github.com/IVDC-Club-IIT-Indore/
IITISoC-25-IVR09.git